

探索使用 nix 扩展 deepin 软件生态

-- rewine

目录

CONTENTS

01 什么是 NIX

02 NIX 和 HM 的安装与使用

03 使用 NIX 管理开发环境

01 什么是 NIX

- nix 包管理器：确保依赖隔离、可重现构建，支持多版本共存。支持 linux 和 darwin
- nix lang：纯函数式语言，用于声明式定义包和系统配置
- NixOS：基于 Nix 的 Linux 发行版，声明式配置系统，支持原子升级、可回滚，不可变
- nixpkgs：包括 nixlang 的准标准库，nixos 模块定义，软件包仓库

Top repositories

By total number of packaged projects

1. nix (nixpkgs unstable) - 105317
2. AUR - 77395
3. Debian+derivs (Raspbian Testing) - 42874
4. FreeBSD Ports - 31502
5. GNU Guix - 30770
6. Fedora (Fedora 40) - 24346
7. Rosa (Rosa 13) - 20003
8. ALT Sisyphus - 19644
9. Gentoo (LiGurOS develop) - 19316
10. MacPorts - 19276

By number of non-unique¹ packaged projects

1. nix (nixpkgs unstable) - 84178
2. AUR - 39817
3. Debian+derivs (Raspbian Testing) - 31781
4. FreeBSD Ports - 26719
5. GNU Guix - 23253
6. Fedora (Fedora 40) - 21892
7. Rosa (Rosa 13) - 19161
8. MacPorts - 17326
9. Gentoo (LiGurOS develop) - 16710
10. ALT Sisyphus - 16154

By number of maintainers

1. AUR - 15786
2. Debian+derivs (Raspbian Testing) - 4231
3. nix (nixpkgs unstable) - 4130
4. FreeBSD Ports - 1665
5. SlackBuilds - 929
6. Spack - 844
7. Gentoo (LiGurOS stable) - 791
8. Void Linux x86_64 - 740
9. Alpine (Alpine Linux Edge) - 674
10. MacPorts - 544

By number of projects per maintainer^{2,3}

1. Debian+derivs (Debian 11 Backports) - 1.9
2. OpenWrt (OpenWrt 24.10 x86_64) - 4.3
3. AUR - 4.9
4. Gentoo overlay GURU - 6.9
5. Spack - 9.9
6. SlackBuilds - 10.3
7. Void Linux x86_64 - 12.0
8. Alpine (Alpine Linux Edge) - 17.3
9. FreeBSD Ports - 18.9
10. Gentoo (LiGurOS stable) - 24.3

By number of projects with up to date packages

1. nix (nixpkgs unstable) - 68647
2. AUR - 24321
3. Debian+derivs (Raspbian Testing) - 20792
4. FreeBSD Ports - 19287
5. Fedora (Fedora Rawhide) - 15986
6. Gentoo (LiGurOS develop) - 11151
7. ALT Sisyphus - 10212
8. openSUSE (openSUSE Tumbleweed) - 10201
9. pkgsrc current - 10146
10. MacPorts - 9884

By percentage of up to date projects²

1. Ravenports - 97.14%
2. Arch+derivs (BioArch) - 95.58%
3. Homebrew - 93.66%
4. IzzyOnDroid - 92.86%
5. KaOS - 92.13%
6. CRUX (CRUX 3.8) - 91.01%
7. stal/IX - 90.86%
8. openmamba - 87.33%
9. Termux - 86.76%
10. nix (nixpkgs unstable) - 86.15%

nix

完美支持命令行应用

支持 Gui 应用，但在 NixOS 之外有风险

软件包靠路径隔离，不依赖容器

软件包数量极多，但不是所有应用在 NixOS 之外正常

玲珑

完全不支持命令行应用

完美支持 Gui 应用跨发行版使用

依赖容器方案

生态建设仍然在早期，大部分在 deepin 经过测试

个人使用 nix 主要用来安装 fd, ripgrep, yazi, ncdu, gh 等 deepin 仓库缺少或者只有旧版本的现代 unix 工具，90%以上都是命令行或者 tui 应用

02 NIX 和 HM 安装

[可选] 建立 /nix 目录

deepin 默认安装 data 分区大，可以考虑修改 /etc/fstab 挂载在 /data 上：

```
/data/nix /nix none defaults,bind 0 0
```

一行命令安装：

```
sh <(curl -L https://mirrors.tuna.tsinghua.edu.cn/nix/latest/install)
```

后面加 --daemon 参数为多用户安装

卸载方法自取：

```
https://nix.dev/manual/nix/2.21/installation/uninstall
```

如果是单用户安装只需要 `$ rm -rf /nix`

修改 ~/.config/nix/nix.conf 配置：

experimental-features = nix-command flakes

substituters = <https://mirrors.tuna.tsinghua.edu.cn/nix-channels/store> <https://cache.nixos.org/>

来试试！

```
test@test-nblkwax9x ~>  
test@test-nblkwax9x ~> nix run nixpkgs#hello  
世界你好！
```

- 搜索一个软件包，不如 <https://search.nixos.org/>

nix search nixpkgs fd

- 临时使用一个包

nix shell nixpkgs#fd

- 进入包的构建环境，仅供开发者使用

nix develop nixpkgs#fd

- 永久安装一个包

nix profile install nixpkgs#fd

- 列举已经安装包

nix profile list

- 删除安装包

nix profile remove fd

- 清理无用的包

nix-collect-garbage

```
test@test-nblkwax9x:~$ fd
fd: 未找到命令
test@test-nblkwax9x:~$ nix shell nixpkgs#fd
test@test-nblkwax9x:~$ fd home.nix
nix-dotfiles/home.nix
test@test-nblkwax9x:~$ exit
exit
test@test-nblkwax9x:~$ fd
fd: 未找到命令
```

nix repl 可以进行交互终端，认识列表和属性集即可

```
nix-repl> 1+1
2

nix-repl> let x=1;y=2;in x+y
3

nix-repl> let x=[1 2];y=[3];in x++y
[
  1
  2
  3
]

nix-repl> let x={x = 1; y = "2"; z = {x = 1;};};in x.z.x
1

nix-repl> let x={x = 1; y = "2"; z = {x = 1;};};in x.y
"2"
```

home-manager 是一个管理用户级安装包和配置文件的项目，支持跨发行版使用。

hm 的安装方法就像“茴”字的4种写法，只推荐一种：

用 hm 安装 hm

```
> programs.home-manager.enable = true;
```

没有安装 hm 如何使用 home-manager ?

```
> nix shell nixpkgs#home-manager
```

从一个模板开始

```
git clone git@github.com:wineeee/nix-dotfiles.git  
cd nix-dotfiles
```

(可选) 更新仓库

```
nix flake update
```

使用 hm 安装 hm

```
nix shell nixpkgs#home-manager  
home-manager switch --flake .
```

```
description = "Home Manager configuration";

inputs = {
  nixpkgs.url = "github:nixos/nixpkgs/nixos-unstable";
  home-manager = {
    url = "github:nix-community/home-manager";
    inputs.nixpkgs.follows = "nixpkgs";
  };
  flake-utils.url = "github:numtide/flake-utils";
  nixgl = {
    url = "github:nix-community/nixGL";
    inputs.nixpkgs.follows = "nixpkgs";
  };
};

outputs = {
  self,
  nixpkgs,
  home-manager,
  flake-utils,
  nixgl,
  ...
}@inputs:
inputs.flake-utils.lib.eachDefaultSystemPassThrough (system: {
  # 将 test 修改为你的用户名
  homeConfigurations.test = home-manager.lib.homeManagerConfiguration {
    pkgs = import nixpkgs {
      inherit system;
      overlays = [ nixgl.overlay ];
    };
  };
  # Specify your home configuration modules here, for example,
  # the path to your home.nix.
  modules = [
    ./home.nix
  ];
  # Optionally use extraSpecialArgs
  # to pass through arguments to home.nix
  extraSpecialArgs = { inherit inputs; };
};
});
```

```
{
  inputs,
  config,
  pkgs,
  ...
}:

{
  # 将 test 修改为你的用户名
  home.username = "test";
  home.homeDirectory = "/home/test";

  home.packages = with pkgs; [
    comma
    manix
    nixgl.nixGLIntel
  ];

  programs.fish.enable = true;

  programs.direnv = {
    enable = true;
    nix-direnv.enable = true;
  };

  home.stateVersion = "24.11";

  # Let Home Manager install and manage itself.
  programs.home-manager.enable = true;
}
```


➤ nix-index

运行一次 nix-index 更新数据库，用 nix-locate 可以根据文件名查找来自哪个包

➤ comma

只需要在命令前加一个 “,” 就能自动查找并下载对应软件包

```
rewine@nixos ~> whereis glxinfo
glxinfo:
rewine@nixos ~> , glxinfo
name of display: :0
display: :0 screen: 0
direct rendering: Yes
server glx vendor string: SGI
server glx version string: 1.4
```

应用无法打开，报错：

libGL error: unable to load driver: i965_dri.so
libGL error: driver pointer missing

常见于使用 GPU 渲染的应用：

1. 尝试使用软渲染，比如 Qt 可以设置 QT_QUICK_BACKEND=software 环境变量
2. 安装 nixGL，比如 qtcreeator 无法打开，改用 nixGL qtcreeator 命令启动

应用提供 systemd service 的无法使用

手动复制一份到 `/usr/lib/systemd/user` ?

系统级管理器比如 nix-maid

和系统耦合深的软件尽可能不要使用 nix 版本

使用 NIX 管理开发环境

以 waylib 为例，演示一下如何使用 nix 和 direnv 管理开发环境：

<https://github.com/vioken/waylib>

Devenv 是对 nix 更高层级的封装，使用更简单，可以提供快速、声明式、可重现和可组合的开发环境。

可以使用 `devenv init` 一键初始化

文档：<https://devenv.sh>

配置 python venv 环境示例：

```
# https://devenv.sh/languages/  
languages.python = {  
  enable = true;  
  venv.enable = true;  
  venv.requirements = ''  
    PySide6  
  '';  
};
```

deepin, 友你更好



deepin微信公众号



deepin论坛微信小程序